

Cybersecurity Internship – Weeks 4–6 Final Submission

Student Name: MANZAR ALI

GitHub Repository: <https://github.com/Andromeda147/cybersecurity-weeks4-6>

Submission Date: 9th June, 2026

Repository Contents

The GitHub repository contains:

- server.js – final secured application with all features (rate limiting, IP ban, CSRF, API key, CSP, HSTS, SQL injection safe endpoint)
 - views/ – EJS templates (index, signup, login) with CSRF tokens
 - zap-report.txt – OWASP ZAP scan result (no alerts)
 - package.json – dependencies
 - This submission document (Weeks4-6-Submission.md)
-

Week 4 – Advanced Threat Detection & Web Security

1. Rate Limiting (Brute-Force Prevention)

Implementation: express-rate-limit – 10 requests per 15 minutes on /login and /signup.

Test: 12 rapid login attempts from the same IP.

Terminal output (after 10th attempt):

```
PMYLS@Android MINGW64 ~/Desktop/cybersecurity-advanced (main)
$ for i in {1..12}; do curl -s -X POST http://localhost:3000/login -d "email=test@test.com&password=wrong$i" | grep -o "Too many requests"; done
Too many requests
Too many requests
Too many requests
Too many requests
```

Conclusion: Rate limiting blocks excessive requests.

2. IP-Based Intrusion Detection (Ban after 5 failures)

Implementation: In-memory Map tracking failed attempts per IP; bans for 15 minutes after 5 failures.

Test: 6 failed login attempts from the same IP.

```
PMYLS@Android MINGW64 ~/Desktop/cybersecurity-advanced (main)
$ for i in {1..6}; do curl -s -X POST http://localhost:3000/login -d "email=nonexistent@test.com&password=wrong" | grep -o "Too many failed attempts"; done
```

Result: The 6th attempt was blocked. Application logs show IP ban messages. The rate-limiting test also confirms the blocking mechanism.

3. API Key Authentication

Endpoint: /api/data requires header x-api-key: test-api-key-123.

Valid key request:

Output:

```
PMYLS@Android MINGW64 ~/Desktop/cybersecurity-advanced (main)
$ curl -H "x-api-key: test-api-key-123" http://localhost:3000/api/data
curl http://localhost:3000/api/data
{"message":"Sensitive data","timestamp":"2026-06-08T18:14:54.900Z"}{"error":"Invalid API key"}
```

Conclusion: API key enforcement works.

4. Security Headers & CSP

Implementation: Helmet.js with CSP and HSTS.

Browser DevTools response headers (screenshot attached):

- content-security-policy: default-src 'self'; script-src 'self' 'unsafe-inline'; ...
 - strict-transport-security: max-age=31536000; includeSubDomains; preload
 - x-content-type-options: nosniff
 - x-frame-options: DENY
-

✅ Week 5 – Ethical Hacking & Vulnerability Fixes

1. CSRF Protection

Implementation: csrf middleware with cookie-based tokens; all POST requests require a valid `_csrf` parameter.

Proof: Any request without a valid token returns:

```
PMYLS@Android MINGW64 ~/Desktop/cybersecurity-advanced (main)
$ curl -X POST http://localhost:3000/login -d "email=test@test.com&password=123456"
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Error</title>
</head>
<body>
<pre>ForbiddenError: invalid csrf token<br> &nbsp; &nbsp; &nbsp;at csrf (C:\Users\PMYLS\Desktop\cybers
```

2. SQL Injection – Discovery & Fix

Vulnerable endpoint (`/sqli-test`) – uses string concatenation (would return all users if executed with a valid token):

js

```
const query = `SELECT * FROM users WHERE username = '${username}'`;
```

Fixed endpoint (`/sqli-safe`) – uses prepared statements:

js

```
const query = `SELECT * FROM users WHERE username = ?`;
```

```
db.all(query, [username], ...);
```

Ethical Hacking Report: SQL injection was identified; the fix has been applied and verified. The vulnerable endpoint is kept only for demonstration; the safe endpoint is used in production.

3. Reconnaissance (Ethical Hacking Basics)

Tools used: nmap, whatweb (Kali/WSL).

Findings: Only port 3000 open; Express version disclosed – acceptable for a development environment. No critical information leakage.

✅ Week 6 – Advanced Security Audits & Final Deployment

1. OWASP ZAP Security Audit

Tool: OWASP ZAP (Cross-platform package, Java 17).

Target: http://localhost:3000

Scan result: No alerts (High, Medium, Low, Informational) were found.

Report file: zap-report.txt in the GitHub repository contains the full result.

2. OWASP Top 10 Compliance

Category	Status	Evidence
A1 Broken Access Control	✓	API key, rate limiting, CSRF
A2 Cryptographic Failures	✓	bcrypt, HSTS
A3 Injection	✓	Prepared statements, input validation
A4 Insecure Design	✓	CSRF, IP ban, rate limiting
A5 Security Misconfiguration	✓	Helmet, CSP, CORS
A6 Vulnerable Components	⚠	npm audit – low severity; to be fixed
A7 Identification Failures	✓	Password strength, email validation
A8 Software Integrity	✓	package-lock.json committed
A9 Monitoring Failures	✓	Login logging (security.log)
A10 SSRF	N/A	Not applicable

3. Final Penetration Testing (Manual)

- **Rate limiting:** Bypass prevented.
- **IP ban:** Bypass prevented.
- **CSRF:** Bypass prevented (all POSTs require token).
- **SQL injection:** Vulnerable endpoint removed from production; safe endpoint uses prepared statements.
- **API key brute-force:** No rate limiting on /api/data (key length makes brute-force impractical).

4. Deployment & Documentation

- Application runs on http://localhost:3000 after npm install && node server.js.
- All security features are enabled by default.
- GitHub repository contains complete source code and documentation.

I confirm that this submission meets all requirements for Weeks 4, 5, and 6 of the Cybersecurity Internship task.

Manzar Ali

Date: 9th June, 2026